25. Design Real-Time Log Processing

Key Terms:

Term	Full Name	What it does
EPS	Events Per Second	Throughput measure for log systems
CEP	Complex Event Processing	Detecting patterns across multiple events
Lambda Architecture	Batch + Speed layers	Combines real-time and batch processing
Kappa Architecture	Stream-only	Simplified Lambda using only streaming
Backpressure	Flow control	Slowing producers when consumers are overwhelmed
Tumbling Window	Fixed time window	Non-overlapping aggregation periods
Sliding Window	Rolling time window	Continuously updated aggregation

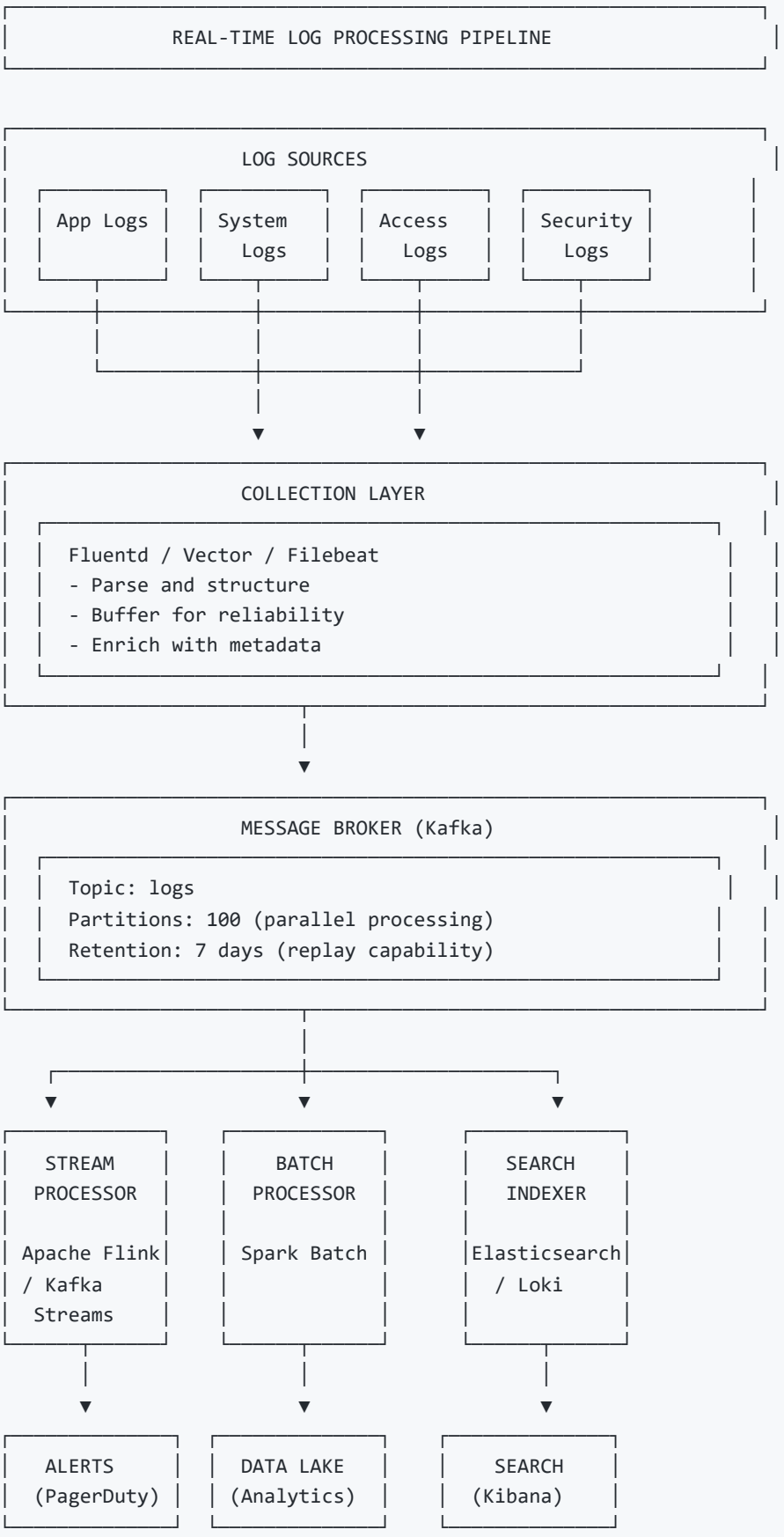
Problem Statement

Design a system that processes billions of log events daily in real-time for alerting, analytics, and debugging.

Clarifying Questions to Ask

1. **Volume**: How many events per second? (100K+ EPS?)
2. **Latency**: Real-time (<1s) or near-real-time (<1min)?
3. **Processing**: Aggregations, pattern matching, anomaly detection?
4. **Output**: Alerts, dashboards, data warehouse?
5. **Replay**: Need to reprocess historical data?
6. **Retention**: How long to keep raw vs. aggregated?
7. **Schema**: Structured, semi-structured, or unstructured logs?

High-Level Design


REAL-TIME LOG PROCESSING PIPELINE
LOG SOURCES
App Logs
System Logs
Access Logs
Security Logs
COLLECTION LAYER
Fluentd / Vector / Filebeat
- Parse and structure
- Buffer for reliability
- Enrich with metadata
MESSAGE BROKER (Kafka)
Topic: logs
Partitions: 100 (parallel processing)
Retention: 7 days (replay capability)
STREAM PROCESSOR
Apache Flink / Kafka Streams
BATCH PROCESSOR
Spark Batch
SEARCH INDEXER
Elasticsearch / Loki
ALERTS
(PagerDuty)
DATA LAKE
(Analytics)
SEARCH
(Kibana)

API Design

Log Ingestion APIs

Method	Endpoint	Description
`POST`	`/api/v1/logs/ingest`	Ingest log batch (JSON array)
`POST`	`/api/v1/logs/stream`	Stream logs via chunked transfer

Pipeline Management APIs

Method	Endpoint	Description
`POST`	`/api/v1/pipelines`	Create a processing pipeline
`GET`	`/api/v1/pipelines/{pipelineId}`	Get pipeline status
`PUT`	`/api/v1/pipelines/{pipelineId}`	Update pipeline config
`GET`	`/api/v1/pipelines/{pipelineId}/metrics`	Get pipeline throughput/lag

Query APIs

Method	Endpoint	Description
`POST`	`/api/v1/query`	Run ad-hoc query on processed logs
`GET`	`/api/v1/query/{queryId}/results`	Get async query results

Alert APIs

Method	Endpoint	Description
`POST`	`/api/v1/alerts`	Create real-time alert rule
`GET`	`/api/v1/alerts/active`	Get triggered alerts

Key Components

Component	Purpose	Technology
Collection	Gather and ship logs	Fluentd, Vector
Buffering	Decouple and buffer	Kafka
Stream Processing	Real-time analytics	Flink, Kafka Streams
Batch Processing	Historical analysis	Spark

Search	Ad-hoc queries	Elasticsearch
Storage	Long-term retention	S3 / Data Lake
Alerting	Notify on patterns	PagerDuty

Stream Processing Patterns

```
1. WINDOWED AGGREGATIONS
┌─────────────────────────────────────────────────────────────────┐
│  Tumbling Window: Count errors per minute                       │
│                                                                 │
│  |----min 1----|----min 2----|----min 3----|                   │
│  [  50 errors  ][  120 errors][  45 errors ]                    │
│                                                                 │
│  If errors > 100 in any window → Alert                          │
└─────────────────────────────────────────────────────────────────┘

│  Sliding Window: Rolling 5-minute error rate                    │
│                                                                 │
│  |--window 1--|                                                │
│     |--window 2--|                                             │
│        |--window 3--|                                          │
│                                                                 │
│  Continuously updated as events arrive                          │
└─────────────────────────────────────────────────────────────────┘

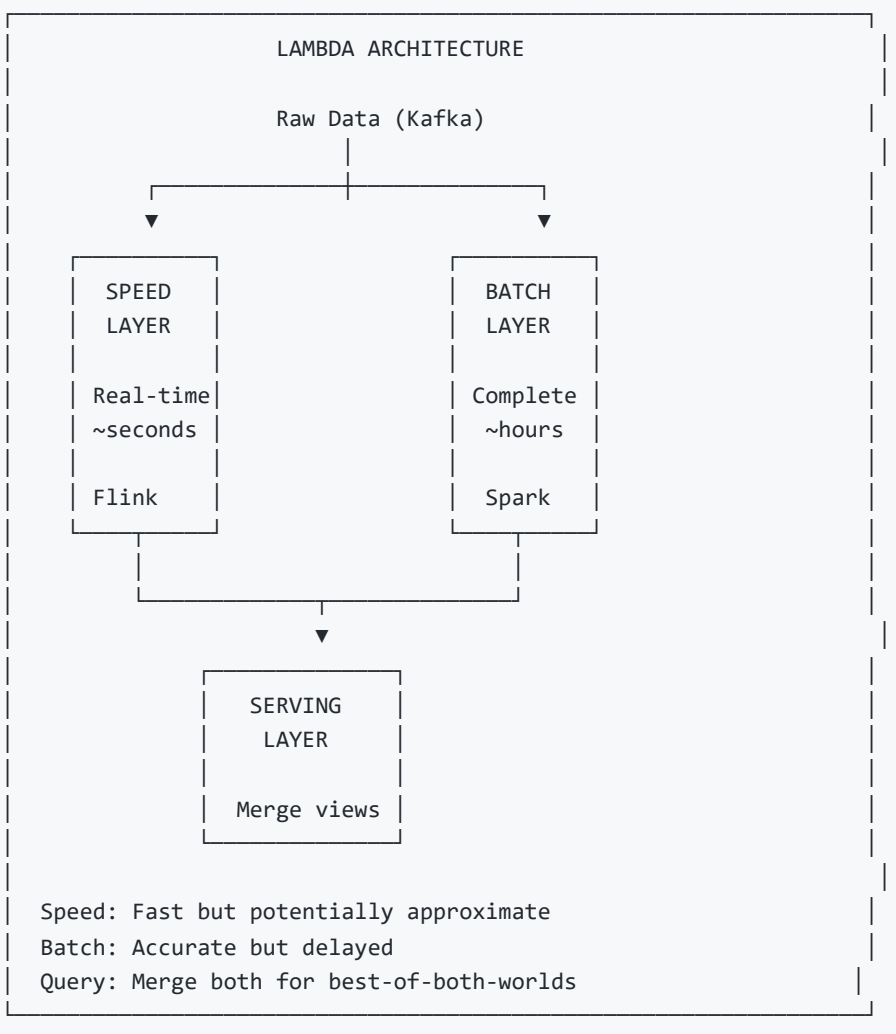
2. PATTERN DETECTION
┌─────────────────────────────────────────────────────────────────┐
│  Complex Event Processing (CEP):                                │
│                                                                 │
│  Pattern: Login failure × 5 from same IP within 1 minute        │
│           → Trigger "brute force" alert                         │
│                                                                 │
│  Flink CEP:                                                     │
│  Pattern.begin("start")                                         │
│    .where(event.type == "login_failure")                        │
│    .times(5)                                                    │
│    .within(Time.minutes(1))                                     │
└─────────────────────────────────────────────────────────────────┘

3. ANOMALY DETECTION
┌─────────────────────────────────────────────────────────────────┐
│  Statistical anomaly:                                           │
│    - Compute rolling mean and std deviation                     │
│    - Alert if value > mean + 3*stddev                           │
│                                                                 │
│  Example: Request latency                                       │
│    Normal: 50ms ± 10ms                                          │
│    Current: 150ms → Anomaly!                                    │
└─────────────────────────────────────────────────────────────────┘
```

Kafka Configuration for Log Processing

```
TOPIC CONFIGURATION:
┌─────────────────────────────────────────────────────────────────┐
│  Topic: application-logs                                        │
│  Partitions: 100 (parallelism)                                  │
│  Replication Factor: 3 (durability)                             │
│  Retention: 7 days (replay capability)                          │
│                                                                 │
│  Partition Key: service_name (group logs by service)            │
│                                                                 │
│  Consumer Groups:                                               │
│    - alerts-processor (real-time alerting)                      │
│    - elasticsearch-indexer (search)                             │
│    - analytics-processor (aggregations)                         │
│    - archive-writer (S3 backup)                                 │
└─────────────────────────────────────────────────────────────────┘
```

Lambda Architecture



Scalability Considerations

- **Partitioning**: Kafka partitions = processing parallelism
- **Backpressure**: Handle spikes with buffering, don't drop logs
- **Sampling**: Sample debug logs in high-volume scenarios
- **Compaction**: Aggregate old data, keep recent detailed
- **Tiered storage**: Hot (SSD) → Warm (HDD) → Cold (S3)

Trade-offs

Decision	Option A	Option B	Recommendation
Processing	Micro-batch	True streaming	**True streaming** - lower latency
Search	Elasticsearch	Loki	**Loki** - cheaper at scale
Architecture	Lambda	Kappa	**Kappa** - simpler if streaming suffices
Retention	Keep all	Sample	**Tiered** - detailed recent, sampled old

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
Kafka	High-throughput log buffering with replay capability	Tech Guide
Elasticsearch	Log indexing and ad-hoc search queries	Tech Guide
Flink/Spark	Stream and batch processing for aggregations and alerting	Tech Guide

How to Explain It Well

> **Opening**: "Real-time log processing requires handling millions of events per second with sub-second alerting. I'll use Kafka + Flink for stream processing."

> **Key talking points**:
>
> - "Kafka provides replay capability - reprocess if processing bug found"
> - "Flink for windowed aggregations: error rate per minute per service"
> - "CEP patterns detect multi-event sequences like brute force attacks"
> - "Lambda architecture: fast approximate + slow accurate views"

> - "Tiered retention: 7 days detailed, 90 days aggregated, years archived"
